

IntentGuard: Stateful Task-Scoped Authorization for AI Agent Trajectories

Prabhu Sivapunniyam
Independent Researcher

Abstract - Tool-using AI agents may propose individually permitted actions whose accumulated effects exceed the user task. Operation allowlists cannot express scoped budgets or required ordering, making this distinction important for repeated access and disclosure. We implement IntentGuard, a trusted runtime that evaluates explicit task contracts against admitted history before executing tools. Its mechanism study isolates per-operation and per-resource budgets, distinct-destination limits, and successful-operation sequences beyond a total action count; it does not claim novelty for counters or finite-state policies themselves. The implementation passes 41 tests. Across 44 synthetic cases with 69 proposals, it matches all expected decisions, executes all 47 authorized proposals, and executes none of 22 proposals requiring blocking or confirmation. A count-only control executes six unauthorized proposals that richer trajectory constraints reject. These results demonstrate deterministic enforcement on constructed inputs, not prompt-injection resistance for a real model, task-completion utility, or superiority over published defenses.

Index Terms: AI agents, task authorization, prompt injection, runtime enforcement, least privilege, trajectory constraints.

I. INTRODUCTION

An AI assistant retrieving an invoice may possess credentials capable of sending messages, modifying records, and reading unrelated documents. The user may nevertheless have authorized only retrieval of one invoice amount. If retrieved content asks the assistant to forward the invoice externally, a capability check may permit the invocation while the action violates the task. We use authorization drift to describe divergence between task authority and proposed operations or cumulative effects.

Task relevance does not by itself establish authority: sending a document can help resolve an invoice question while violating a recipient restriction. Conversely, an unfamiliar resource can be necessary for a legitimate search. An

enforcement system must expose which boundaries are hard constraints and which require confirmation. Treating uncertainty as permission risks disclosure; rejecting every uncertainty can prevent useful work.

IntentGuard evaluates a normalized proposal before execution and returns ALLOW, CONFIRM, or BLOCK with a reason. A trusted tool registry declares required operands and side effects. A task lock serializes authorization, charging, and bounded execution so concurrent calls cannot independently consume the same remaining budget. The human-readable objective is stored but not interpreted semantically; contracts are supplied as structured data to separate enforcement errors from natural-language compilation errors.

The study asks what violations each policy mechanism prevents under matched proposals and what legitimate actions it obstructs. The contribution comprises an executable contract, a guarded in-memory runtime, 21 synthetic pairs, and a reproducible five-policy comparison. The results isolate static boundaries, scoped budgets, and ordered execution. The planner proposes; the trusted runtime authorizes. A full agent study remains necessary for adaptive attacks, ambiguous tasks, and generalization beyond the selected inputs.

II. RELATED WORK AND RESEARCH GAP

A. Evaluation and task alignment

AgentDojo [1] and InjecAgent [2] provide environments for studying indirect prompt injection and tool-using agents. Their scope is broader than the fixed proposals used here; percentages from these datasets cannot be ranked against our diagnostic counts. Task Shield [3] checks whether instructions and calls contribute to the user task. This overlaps with a proposed semantic layer, whereas the present design exposes deterministic constraints for independent inspection and replay.

B. Authorization, provenance, and trajectories

DRIFT [4] combines a planned function trajectory, parameter constraints, dynamic validation, and instruction isolation. IntentGuard enforces explicit budgets and a fixed operation sequence, not equivalent plan reasoning or memory isolation. PAuth [5] derives task-scoped specifications and binds values to signed provenance; it directly addresses operator permission versus authority for a concrete operation. IntentGuard neither originates task-scoped authorization nor provides PAuth's provenance guarantee. CaMeL [6] separates control and data flows and constrains tool calls through capabilities. Explicit resource and destination sets alone cannot provide its information-flow guarantees.

These approaches already address task alignment, authorization, provenance, or trajectories. We do not assert that none addresses stateful authorization. The gap examined here is an ablation question: under identical tool semantics and proposals, which explicit history constraints reject violations that a total count permits? A resource may be accessed too often while the global budget remains available, and a send may occur before a required draft. Published defenses are contextual comparisons, not implemented baselines; assessing incremental research novelty requires faithful external comparisons.

III. SYSTEM MODEL AND METHODOLOGY

A. Trust boundary and representation

The model contains a user, an untrusted or fallible planner, a trusted contract authority, a guarded runtime, and bounded tools. The host owns the runtime and tool credentials. The planner supplies proposals, not contract updates, approvals, or history. Attacker-controlled tool content and planner mistakes are in scope; compromise of the host, runtime, or tool adapter is excluded. All evaluated effects are synthetic and remain in memory.

Let $C = (O, R, D, P, F, K)$ denote allowed operations, resources, destinations, prohibited operations, confirmation-required effects, and an optional count limit. A proposal $a = (o, r, d, s)$ contains operation, resource, optional destination, and optional effect. The runtime validates

operands and derives s from trusted tool metadata. Extend C with operation budgets B_o , resource budgets B_r , an optional ordered sequence Q , and a distinct-destination bound U . State H stores admitted counts, destination set, and the successful sequence index. Counts include failed invocations; sequence progress does not. Hard constraints are evaluated before confirmation; only ALLOW reaches a tool.

B. Worked task contract

Consider: "Read invoice 123. Ask before sending it to me. Do not access other documents." Listing 1 encodes this task with a two-invocation budget. The registered send adapter requires a destination and declares external_communication. Omitting that effect from the proposal cannot suppress confirmation.

Listing 1. Worked task authorization contract

```
{
  "objective": "Read invoice; confirm sending",
  "allowed_operations": ["read", "send"],
  "allowed_resources": ["invoice_123"],
  "allowed_destinations": ["user@example.com"],
  "prohibited_operations": ["delete"],
  "confirmation_required": ["external_communication"],
  "max_action_count": 2
}
```

TABLE I. WORKED CONTRACT OUTCOMES

Separate proposal	History h	Decision
Read invoice_123	0	ALLOW
Read payroll	0	BLOCK
Send to external address	0	BLOCK
Send to user@example.com	0	CONFIRM
Read invoice_123	2	BLOCK

Table I treats each row as a separate proposal. Rows other than the exhausted-budget example begin at $h = 0$; the last row begins after two admitted reads. The contract authorizes read attempts but does not automatically approve sending. A valid recipient yields CONFIRM, whereas an external recipient violates a hard boundary and yields BLOCK. CONFIRM suspends execution; authenticated approval and resumption are not implemented.

IV. ARCHITECTURE AND AUTHORIZATION ALGORITHM

A. Guarded execution boundary

Figure 1 distinguishes the proposed compilation stage from implemented enforcement. Natural-language intent would be compiled and validated by the host or user before installation. Current experiments supply structured contracts manually. Validated frozen objects are immutable data, not signed authority. The trusted registry and task lock mediate proposals against runtime-owned history. A process boundary remains necessary if planner code itself is adversarial.

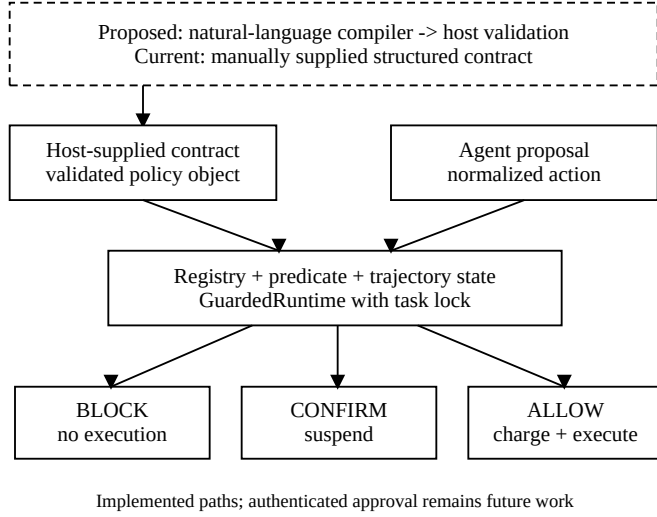


Fig. 1. Proposed compilation (dashed) and implemented execution boundary. Trusted registry metadata and a task lock mediate mock tools. CONFIRM suspends; authenticated approval and resumption remain future work.

The wrapper rejects unknown adapters, missing required destinations, unexpected destination fields, and conflicting side-effect declarations. It derives omitted effects from trusted metadata before calling the predicate. Explicitly prohibited operations take precedence over an allowlist entry. Operation, resource, destination, budget, and sequence violations take precedence over confirmation. Figure 2 summarizes the resulting decision flow.

B. Execution algorithm

Every ALLOW charges global, operation, and resource counts and records its destination before the handler runs. Only a successful handler advances the sequence; reentrant execution is rejected. A tool exception is retained in the result and does not refund the charge. BLOCK and CONFIRM produce trace records without tool execution or history consumption. Malformed contracts and proposals raise validation errors before execution. Decision and outcome traces are neither durable nor tamper-evident.

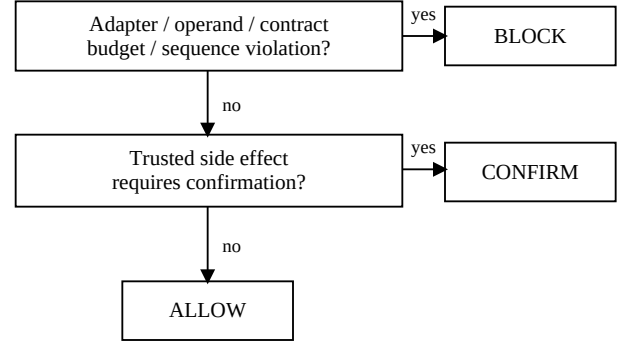


Fig. 2. Runtime decision flow after proposal validation. Missing required destinations and hard violations block before confirmation; side effects come from trusted metadata.

Algorithm 1. Guarded execution with shared history

Input: contract C, proposal a, trusted registry T
State: counts, destinations, sequence index; lock L
1 Validate proposal; acquire L; reject reentrancy
2 Derive effects and validate adapter operands
3 Check static boundaries and global count
4 Check operation/resource budgets, destinations, and next successful-sequence operation
5 Apply confirmation after hard constraints
6 If BLOCK or CONFIRM: trace; return
7 Charge all budgets; record destination
8 Execute handler; retain result or exception
9 Advance sequence only if handler succeeds
10 Trace outcome; release L on every exit

C. Cumulative safety and concurrency

For a finite limit K and an initial count of zero, every admission requires $h < K$ and increments h before invoking a handler. Since one task lock serializes this transition, induction bounds admitted invocations by K , including failed invocations. The same argument applies to each scoped budget. Under the lock, an admitted operation must equal the next sequence symbol; success advances exactly one position. This property assumes a trusted host and a shared runtime instance. It does not extend to independent processes with separate counters, direct tool calls that bypass the wrapper, or authenticated multi-agent delegation.

The implementation holds the lock across bounded mock execution. This simplifies accounting and prevents a check-then-execute race, but can serialize slow handlers in a deployment. A future reservation-based scheduler must preserve the same count invariant while handling cancellation, retries, and persistence. Confirmation similarly needs a trusted approval record bound to the task, operands, and contract version; a planner-supplied approval flag is not accepted.

V. EXPERIMENTAL EVALUATION

A. Cases and local controls

The benchmark retains 15 paired designs P01-P15 and two original fixtures, and adds six trajectory pairs T01-T06. The resulting 44 cases contain 69 proposals: 47 ALLOW, 20 BLOCK, and two CONFIRM. New pairs isolate repeated resource access, repeated sends, ordering, destination cardinality, cross-operation resource use, and replay after sequence completion. Each drift member stays within its total count. Labels are authored before execution, without independent review.

Five policies receive the same proposals and trusted tool metadata. No enforcement removes task restrictions. Operation only retains the operation allowlist. No history retains static boundaries and confirmation but removes all history constraints. Count only adds the global count while omitting richer trajectory policy. IntentGuard enables every constraint. These local controls isolate mechanisms; they do not reproduce Task Shield, DRIFT, PAuth, or CaMeL.

Each case starts with a fresh runtime and zero charged actions. Its complete proposal sequence is replayed, including proposals following a denial. Handlers append synthetic effects to an in-memory list; no files, messages, or financial systems are changed. The original injection fixture's untrusted text is metadata, not input to an LLM. Consequently, the study evaluates fixed action proposals, not whether an attack successfully induces a model to generate them.

B. Outcomes and implementation verification

We count decisions matching the predefined labels, cases with every decision correct, and authorized versus unauthorized mock executions. A proposal labeled CONFIRM is considered unauthorized to execute before approval. Authorized execution measures whether the mock handler was called for an ALLOW-labeled proposal; it does not measure completion of a user's natural-language task. Baseline disagreements are retained as results rather than treated as runner failures.

The 41-test suite covers the original deterministic checks, input validation, destination and confirmation behavior,

hard-block precedence, execution suppression, failed-call accounting, concurrency, and benchmark-runner regressions. A concurrency test submits 40 calls through eight worker threads to one runtime with a limit of two. Additional tests exercise scoped budgets, ordered success, failed-step accounting, distinct destinations, replay rejection, reentrancy, immutable history, and concurrent operation-budget admission. Tests also verify that omitted side effects cannot bypass confirmation and incorrect benchmark labels cause a nonzero runner exit.

C. Whole-runtime timing protocol

Each measured workflow performs three mock calls: read, draft, and send. Conditions include direct handlers, a static contract runtime with a total count, and full trajectory runtimes with 1, 10, 100, 1,000, or 10,000 allowed resources. Contracts and proposals are preconstructed; runtime initialization, operand checks, locking, history, traces, and mock execution are timed together. This is local workflow latency, not model inference or network latency.

Each condition receives 100 warmups and 2,000 measured workflows. Condition order is shuffled each round with seed 20261002. We report linearly interpolated p50, p95, and p99 workflow latency. The 14,000 measured workflows comprise 42,000 calls, not independent security cases. Measurements were collected October 2, 2026, using CPython 3.12.14 on Windows 11 build 26200, Intel64 Family 6 Model 189. Background activity was not isolated; tail comparisons are descriptive.

D. Reproduction and evidence retention

The runner retains expected and actual decisions, effects, and normalized UTF-8 source hashes. This revision uses `trajectory_benchmark_results.json` and `workflow_timing_results.json`, including raw workflow samples. The earlier predicate snapshots remain historical evidence and do not support the new runtime measurements. Tests and benchmark execution are configured for continuous integration. No statistical significance or population-level attack-success claim follows from these fixed cases.

VI. RESULTS AND ANALYSIS

A. Benefits observed in fixed-proposal replay

All 41 automated tests pass. IntentGuard matches 69/69 decisions across 44 cases, executes 47/47 authorized proposals, blocks 20 proposals, and suspends two for confirmation. Table II reports decision agreement, unauthorized execution, and authorized execution. No policy reports mock tool errors.

TABLE II. DECISION AGREEMENT AND MOCK EXECUTIONS

Policy	Agreement	Unauth.	Auth.
No guard	47/69	22	47/47
Operation only	52/69	17	47/47
No history	62/69	7	47/47
Count only	63/69	6	47/47
IntentGuard	69/69	0	47/47

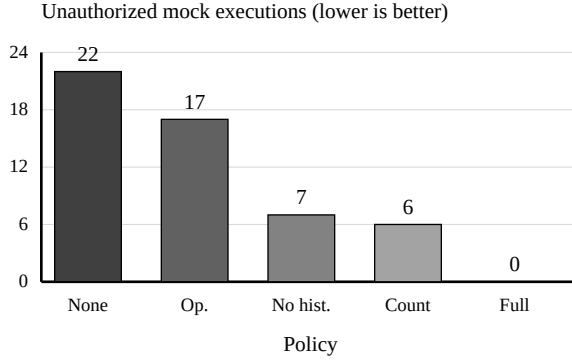


Fig. 3. Unauthorized mock executions among 22 BLOCK- or CONFIRM-labeled proposals. Full denotes IntentGuard. All policies execute the same 47 authorized proposals.

The full policy prevents 17 unauthorized executions admitted by operation-only enforcement. No-history admits seven; count-only admits six. Those six distinguish richer state from a total counter: repeated resource access, repeated sends, premature send, a second destination, cross-operation access, and a completed-sequence replay. All controls preserve the same 47 authorized proposals. This is mechanism coverage on authored cases, not independent evidence of broad robustness.

B. Whole-runtime cost

For one allowed resource, the full three-call workflow has p50/p95/p99 latency of 17.9/26.6/45.9 microseconds, versus

16.4/24.9/52.0 for the static runtime and 0.4/0.6/0.8 for direct handlers. The difference in medians is 1.5 microseconds between full and static conditions; the reversed p99 ordering does not establish an advantage.

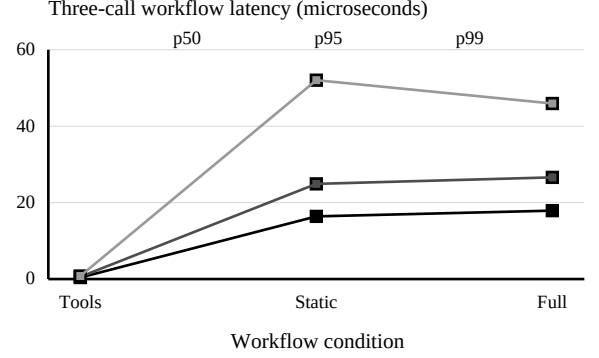


Fig. 4. Local three-call workflow percentiles over 2,000 samples per condition: p50 (black), p95 (dark gray), p99 (light gray). No LLM or network is involved.

C. Resource scaling and interpretation

Across 1-10,000 allowed resources, full-runtime p50 spans 17.9-18.0 microseconds and p95 spans 26.3-27.4. No monotonic increase appears. Policy construction is excluded, and the workload uses one resource, short histories, and immediate handlers. These results cannot predict large-history or distributed-system cost.

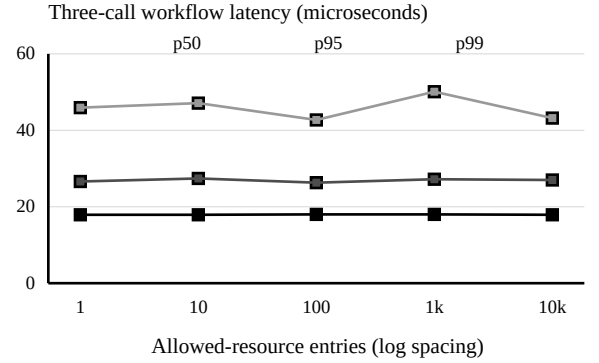


Fig. 5. Whole-runtime resource-set scaling, with p50 (black), p95 (dark gray), and p99 (light gray). Contracts are preconstructed; histories remain short.

No iterative optimizer is used, so a convergence claim would be inapplicable. The figures instead report latency percentiles and scaling. Real agent completion, attack success, confirmation burden, token usage, and model-loop latency remain unmeasured.

VII. LIMITATIONS

A. Evidence and construct validity

The 44 cases exercise known mechanisms and are not a random or held-out task sample. Labels have not received independent review, and no adaptive attacker or LLM generates proposals. The paired cases explain why particular predicates matter, but a perfect result does not establish population-level precision, recall, or attack success. Preserving 47 authorized mock executions is not evidence of natural-language task completion.

The controls are deliberately limited. Operation-only enforcement cannot inspect resources or recipients, and count-only enforcement cannot express scoped budgets or ordering. Their failures on such inputs follow from their definitions. These comparisons isolate mechanisms, not a state-of-the-art ranking. Future studies must include realistic discovery tasks where strict resource enumeration can block legitimate work, along with ambiguous requests and independently reproduced published defenses.

B. Trust and implementation boundary

Contracts are supplied by the experimenter, avoiding natural-language ambiguity and compilation errors. The guarded runtime owns trajectory state and a tool registry within one process, but does not authenticate users, persist state, isolate malicious Python code, or prevent a compromised host from invoking tools directly. Frozen dataclasses and a lock are implementation mechanisms, not cryptographic or operating-system security boundaries.

The registry defines destination requirements and effects for each supported adapter. Incorrect metadata or an adapter that uses undeclared operands can invalidate enforcement. Pure predicate calls outside the wrapper still rely on caller-supplied history and action fields. CONFIRM suspends execution but cannot currently accept and authenticate a user's approval. Durable, operand-bound approval and contract updates remain necessary for a complete interactive workflow.

C. Measurement and generalization

The runtime measurement covers a three-call authorized path on one machine with preconstructed contracts. Interpreter behavior, caches, scheduling, and background load influence its percentiles. The concurrency test verifies a count property for one scenario; it is not a concurrency performance benchmark. Broader measurements should vary action mix, misses, key lengths, policy construction, logging, persistence, and handler latency, reporting tail and end-to-end costs separately.

Resource budgets count admitted calls across operations, not bytes or aggregate value. A destination-cardinality bound

does not track data composition, and a fixed sequence is not semantic plan verification. No semantic judge, provenance system, aggregate-value budget, disclosure-composition policy, or authenticated delegation mechanism is evaluated. The remaining P16-P20 designs require these additional capabilities. The system therefore cannot claim protection against general information-flow attacks, cross-agent authority laundering, or arbitrary multi-step prompt injection. Statistical claims require appropriately sampled tasks and a justified analysis, not additional repetitions of fixed deterministic inputs.

VIII. CONCLUSION AND FUTURE WORK

IntentGuard enforces explicit task authority against admitted history. It passes 41 tests and matches 69/69 decisions on 44 synthetic cases, preserving 47 authorized executions with zero unauthorized executions. Six drift trajectories remain admissible under a total count but are rejected by scoped budgets, destination cardinality, or successful-operation ordering. This supports a deterministic mechanism claim.

The next study should use 50 distinct tasks with benign and injected contexts, yielding 100 task-context conditions, and real model-generated proposals. This is a proposed study, not a completed experiment. Compilation errors must be evaluated separately, labels independently reviewed, and published baselines faithfully reproduced under matched tool and attack budgets. Authenticated approvals must bind task, exact operands, and contract version, with single-use redemption, expiration, substitution, and concurrency tests. Meaningful novelty and practical agent benefit remain to be demonstrated.

REFERENCES

- [1] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramer, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. doi: 10.52202/079017-2636.
- [2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of ACL*, 2024, pp. 10471-10506. doi: 10.18653/v1/2024.findings-acl.624.
- [3] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," in *Proc. ACL*, 2025, pp. 29680-29697. doi: 10.18653/v1/2025.acl-long.1435.
- [4] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, "DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. doi: 10.52202/085713-2791.
- [5] R. K. Sharma, L. Jiang, S. Chen, and Z. Lin, "Beyond OAuth: Task-Scoped Authorization for AI Agents via Natural Language Slices," *arXiv:2603.17170v2*, Aug. 2026. doi: 10.48550/arXiv.2603.17170.
- [6] E. Debenedetti et al., "Defeating Prompt Injections by Design," *arXiv:2503.18813v2*, Jun. 2025. doi: 10.48550/arXiv.2503.18813.